AgentShield AI: A Real-Time Runtime Security Framework for Autonomous AI Agents

Abhay Chauhan, Aditya Sharma, Avnish Kumar, Krishti Mittal
Department of Computer Science & Engineering, IMS Engineering College, Ghaziabad
Dr. A.P.J. Abdul Kalam Technical University, Lucknow

Abstract

Autonomous AI agents built on Large Language Models (LLMs) are increasingly deployed to plan, reason, and act on behalf of users by invoking external tools, APIs, and enterprise data sources. This growing autonomy introduces a class of security risks that conventional cybersecurity mechanisms are not designed to address: prompt injection, jailbreak attacks, unauthorized tool invocation, and sensitive data leakage. Existing defenses in the literature, such as prompt-injection detectors, privilege-based access control, and customizable runtime policy engines, each address a narrow slice of this problem in isolation and typically stop short of providing a single, unified, real-time, and auditable protection layer. This paper proposes AgentShield AI, a runtime security framework that wraps around an existing AI agent to (i) analyze incoming prompts before they reach the agent, (ii) continuously monitor the agent's intermediate reasoning and tool-call behavior, (iii) compute a dynamic, hybrid risk score that combines LLM-based semantic judgment with deterministic rule-based checks, (iv) enforce security policies through a rule-based engine that deterministically allows, flags, or blocks actions, and (v) maintain a complete, explainable audit trail for governance and compliance. We formally define the threat model and problem formulation, describe the architecture and algorithm of each pipeline stage, and lay out an evaluation plan against established prompt-injection benchmarks and baseline defenses. Unlike prior work, which treats detection, access control, and enforcement as separate concerns, AgentShield AI unifies them into a single pipeline positioned between the user and the agent's action space, so that a request is analyzed, scored, and gated before it is ever allowed to execute.

Keywords: *AI agent security, prompt injection, jailbreak detection, LLM runtime monitoring, policy enforcement, autonomous agents, AI governance, risk scoring*

I. INTRODUCTION

Autonomous AI agents integrate a Large Language Model (LLM) with the ability to reason over multi-step tasks and to act on that reasoning by calling external tools, APIs, databases, and enterprise systems [1], [5]. Frameworks such as ReAct [5] formalize this reason-then-act loop, enabling an agent to interleave chain-of-thought reasoning with concrete actions rather than producing a single static response. This shift from passive text generation to autonomous action is what makes modern AI agents useful in practice, and it is also precisely what places them outside the threat model that conventional application security was designed for.

Unlike a traditional application, whose attack surface is defined by its code and its exposed interfaces, an AI agent's attack surface is defined largely by natural language: any text the agent processes, whether typed directly by a user or embedded in a webpage, a document, or a tool's output, is a potential vector for manipulating the agent's behavior. An attacker does not need to exploit a software vulnerability; a carefully crafted sentence hidden in otherwise ordinary data is sufficient to redirect the agent toward an attacker-chosen action [6].

A. Motivation

AI agents are particularly vulnerable to four classes of threats. First, prompt injection, where an attacker embeds an instruction inside data the agent processes, causing the agent to follow the injected instruction instead of its intended task [6], [7]. Second, jailbreaking, where the attacker's input is crafted to bypass the model's built-in safety alignment. Third, unauthorized tool or API invocation, where a manipulated agent is induced to call a tool, endpoint, or data source it should not have access to for the current task. Fourth, sensitive data leakage, where the agent's output inadvertently discloses confidential information it had access to while completing a task. Existing cybersecurity mechanisms, such as static firewalls, access-control lists, and signature-based intrusion detection, are built to reason about packets, code, and known exploit signatures; they have no mechanism for reasoning about the intent behind a natural-language instruction, and therefore cannot reliably detect or prevent these attacks.

B. Limitations of Existing Work

A growing body of work addresses pieces of this problem. Detection-based approaches such as DataSentinel [2] fine-tune a detector LLM to flag contaminated inputs with low false-positive and false-negative rates. Access-control

approaches such as Progent [4] restrict which tools an agent may invoke through privilege policies. Runtime-enforcement approaches such as AgentSpec [3] allow developers to specify custom safety rules that are checked as the agent executes. Each of these is effective at the specific sub-problem it targets, but none of them, individually, provides continuous behavioral monitoring, automated hybrid risk scoring, deterministic enforcement, and compliance-grade audit logging within a single deployed system. In practice, an organization wishing to secure an AI agent today would need to stitch several such point solutions together, with no shared risk model or audit trail between them.

C. Contributions

This paper makes the following contributions:

- We formalize the AI agent runtime-security problem, defining a threat model that captures prompt injection, jailbreaking, unauthorized tool use, and data leakage under a single formulation (Section III).
- We propose AgentShield AI, a five-stage runtime security pipeline: Prompt Analyzer, Behavior Monitor, hybrid Threat Detection and Risk Scoring, a deterministic Policy Enforcement Engine, and a Secure Tool/API Gateway, backed by an Audit Logging and Dashboard layer (Section IV).
- We define a hybrid risk-scoring function that combines LLM-based semantic confidence with deterministic rule-based flags, and a corresponding enforcement decision rule that is not itself susceptible to prompt-based manipulation (Section IV-D and IV-E).
- We lay out a concrete evaluation plan, including benchmark datasets, attacks, baselines, and metrics, against which the completed implementation will be assessed (Section V).

II. RELATED WORK

A. Prompt Injection and Jailbreak Attacks

Prompt injection attacks embed an injected instruction and injected data into the target data processed by an LLM-integrated application, causing the backend model to perform an attacker-chosen injected task instead of the intended target task [6], [7]. These attacks are broadly categorized as heuristic-based, which rely on manually crafted separators such as context-ignoring phrases or fake task-completion text, and optimization-based, which use gradient-guided search to construct adversarial separators that evade a given detector [7]. Jailbreak attacks pursue a related but distinct goal: rather than substituting the task, they aim to bypass the model's safety alignment so that it produces content it would otherwise refuse to generate.

B. Detection-Based Defenses

Known-answer detection concatenates a secret-bearing detection instruction with the target data and checks whether a detector LLM still reproduces the secret; failure to do so signals contamination [7]. DataSentinel [2] improves on this by fine-tuning the detector LLM under a minimax game against simulated adaptive attacks, substantially reducing false-positive and false-negative rates relative to a non-fine-tuned known-answer detector. These approaches are effective at classifying a single piece of input text as clean or contaminated, but they operate upstream of the agent and do not track what the agent subsequently does with a tool or API once execution begins.

C. Access-Control and Runtime-Enforcement Defenses

Progent [4] takes a complementary approach, restricting an agent's tool usage through developer-defined privilege policies, so that even a successfully manipulated agent cannot call tools outside its granted privileges. AgentSpec [3] generalizes this idea into a customizable runtime-enforcement layer in which developers specify safety rules that are checked as the agent executes, independent of whether the manipulation was ever explicitly detected. Both approaches are enforcement-first rather than detection-first, but both rely on manually authored policies or rule sets rather than an automated, learned risk assessment.

D. Traceability and Benchmarking

TraceAegis focuses on reconstructing and auditing the provenance of an agent's tool-call chain after the fact, which is valuable for incident investigation but does not by itself prevent an unauthorized action at the moment it is requested. AgentDojo, presented at NeurIPS 2024, instead provides a standardized benchmark environment in which the robustness of agents and defenses to prompt injection can be measured consistently, and is used in this work as a reference point for the evaluation plan in Section V.

E. Summary of the Gap

Table I summarizes the surveyed work. Across this body of literature, a consistent pattern emerges: each system addresses one stage of the pipeline, either detecting a contaminated input, restricting privileges, enforcing a hand-written rule, or reconstructing provenance after the fact, in isolation. None of the surveyed systems combine real-time prompt analysis, continuous behavioral monitoring, automated hybrid risk scoring, and deterministic, pre-execution policy enforcement within a single unified and auditable framework, which is the gap AgentShield AI is designed to fill.

Work	Year	Focus	Limitation relative to AgentShield AI
OWASP Top 10 for LLM Applications [1]	2025	Risk taxonomy for LLM applications, including prompt injection and insecure output handling.	Defines risks; provides no implementation or enforcement framework.
DataSentinel [2]	2025	Game-theoretic fine-tuning of a detector LLM to flag prompt-injected inputs.	Addresses detection only; no continuous behavior monitoring or action-level enforcement.
Progent [4]	2025	Privilege-based access control restricting AI agent tool usage.	Requires manual privilege policy configuration; no automated risk scoring.
AgentSpec [3]	2025	Customizable runtime policy enforcement for LLM agents.	Enforcement is rule-based only; does not incorporate LLM-based semantic risk assessment.
TraceAegis	2024	Provenance-based tracing and auditing of agent tool-call chains.	Focused on post-hoc traceability rather than real-time, pre-execution blocking.
AgentDojo (NeurIPS 2024)	2024	Benchmark environment for evaluating agent robustness to prompt injection.	An evaluation benchmark, not a deployable defense.
ReAct [5]	2023	Reason-and-act loop underlying most modern agent architectures.	No built-in security or threat-monitoring layer.
GPT-4 Technical Report [8]	2023	Architecture, capabilities, and general safety evaluation of GPT-4.	General-purpose model-level safety evaluation, not agent-specific runtime security.

III. PROBLEM FORMULATION

A. System Model

We consider an AI agent built around a backend LLM f that receives a user prompt x, reasons over a sequence of intermediate steps, and may invoke a set of tools/APIs T = {t1, t2, ..., tn} available to it, such as web search, code execution, database queries, or third-party service calls. We denote by a1, a2, ..., am the sequence of actions (tool/API invocations) the agent attempts to execute while completing a given task.

B. Threat Model

Attacker's goal: an attacker aims to cause the agent to perform an attacker-chosen action or to disclose attacker-desired information, either by contaminating the user prompt x, by embedding an injected instruction in external data the agent consumes (e.g., a webpage, a document, or a tool's output), or by crafting a jailbreak input designed to bypass the backend LLM's safety alignment.

Attacker's capabilities: consistent with prior work on prompt injection [6], [7], we assume the attacker can arbitrarily manipulate any data source that is external to the deployer, such as third-party web content or documents the agent is asked to process, but cannot directly modify the agent's system instructions, the AgentShield AI pipeline itself, or its policy configuration.

Attacker's background knowledge: we consider a strong attacker who may have black-box or, in an adaptive setting, white-box knowledge of the backend LLM, but who does not have access to the internal state of the deterministic Policy Enforcement Engine described in Section IV-E, since that engine's decisions are not derived from a model an attacker could probe or fine-tune against.

C. Defender's Goal

Given a prompt x and the sequence of actions {a1, ..., am} an agent attempts while processing x, the defender's goal is to compute a risk score R(x, ai) for each attempted action and to decide, before execution, whether ai should be allowed, flagged for review, or blocked, such that the true-positive rate for malicious actions is maximized while the false-positive rate on benign actions is minimized, analogous in spirit to the FPR/FNR objective used to evaluate prompt-injection detectors [2], [7], but applied continuously across the agent's action stream rather than once at the input stage.

IV. PROPOSED FRAMEWORK: AGENTSHIELD AI

A. Overview

AgentShield AI is designed as a runtime security layer that wraps around an existing AI agent rather than modifying the agent's own weights or prompts, so that it can be deployed alongside any LLM-based agent with minimal integration effort. The pipeline consists of six stages: (1) Prompt Analyzer, (2) Behavior Monitor, (3) Threat Detection and Risk Scoring, (4) Policy Enforcement Engine, (5) Secure Tool/API Gateway, and (6) Audit Logging and Dashboard. Figure 1 (see accompanying presentation material) illustrates the pipeline as a serpentine flow from prompt to logged, enforced action.

B. Prompt Analyzer

Every incoming prompt x is first passed through the Prompt Analyzer before it reaches the backend agent. The analyzer combines pattern-based heuristics, targeting known separator patterns such as context-ignoring phrases, escape characters, and fake-completion text [7], with LLM-based zero-shot classification, in which a lightweight prompting-based classifier is asked to judge whether x contains an embedded instruction inconsistent with the user's stated task. This mirrors the two complementary strategies used in prior detection work: heuristic pattern matching and LLM-based judgment [2], [7], but here the output is a continuous suspicion signal sP(x) rather than a binary label, so that it can be combined with other signals downstream.

C. Behavior Monitor

As the agent reasons and acts, the Behavior Monitor observes its intermediate reasoning trace and each attempted action ai, comparing it against the profile of actions expected for the current task context (for example, a summarization task requesting a filesystem write is anomalous). The Behavior Monitor produces a per-action suspicion signal sB(ai) that reflects how far the requested action deviates from the expected action profile for the task the agent was given.

D. Threat Detection and Hybrid Risk Scoring

The Prompt Analyzer's signal sP(x) and the Behavior Monitor's signal sB(ai) are combined into a single dynamic risk score for each attempted action:

$$R(x, ai) = \lambda 1 \cdot sP(x) + \lambda 2 \cdot sB(ai) + \lambda 3 \cdot sRULE(ai)$$

where sRULE(ai) is a deterministic indicator that is 1 if ai matches a known high-risk pattern (for example, a request for credentials, an attempt to invoke a restricted tool, or an attempt to exfiltrate data to an external endpoint) and 0 otherwise, and $\lambda 1, \lambda 2, \lambda 3$ are weighting hyperparameters with $\lambda 1 + \lambda 2 + \lambda 3 = 1$. This hybrid formulation follows the same motivation as DataSentinel's use of a fine-tuned detector alongside rule-based known-answer checks [2]: an LLM-based signal alone can be probabilistic and, in principle, adversarially targeted, so a deterministic rule-based term is retained explicitly so that known high-risk patterns are never solely dependent on a model's judgment.

E. Policy Enforcement Engine

Given the computed risk score R(x, ai) and a configurable pair of thresholds τblock and τflag (with τblock > τflag), the Policy Enforcement Engine applies the following deterministic decision rule before ai is permitted to reach the Secure Gateway:

$$Decision(ai) = block \text{ if } R(x,ai) \geq \tau block;\ flag\text{-}for\text{-}review \text{ if } \tau flag \leq R(x,ai) < \tau block;\ allow \text{ if } R(x,ai) < \tau flag$$

Because this engine is rule-based rather than model-based, its decision boundary is fixed and auditable independent of any single inference call, so it is not itself susceptible to the kind of adaptive, gradient-optimized prompt manipulation that has been shown to evade purely model-based detectors [2].

F. Secure Tool/API Gateway

All tool and API calls issued by the agent are required to route through the Secure Gateway, which is the only component with the authority to actually execute an action against an external system. Actions marked allow by the Policy Enforcement Engine are executed normally; actions marked flag-for-review are held pending a lightweight approval step; actions marked block are rejected and never reach the external tool or API, ensuring that a request which evades upstream analysis still cannot bypass the enforcement layer.

G. Audit Logging and Dashboard

Every prompt, intermediate suspicion signal, computed risk score, and enforcement decision is written to an append-only audit log. A dashboard renders this log in an explainable form, showing, for a given action, which of sP, sB, or sRULE contributed most to its risk score, which supports incident review, governance reporting, and compliance auditing of the deployed agent.

H. Pipeline Algorithm

Algorithm 1 summarizes the end-to-end pipeline executed for each user prompt.

Algorithm 1: *AgentShield AI Runtime Pipeline*

Input: user prompt x, agent f, tool set T, thresholds τflag, τblock

```
sP ← PromptAnalyzer(x)
for each action ai attempted by f while processing x do
    sB ← BehaviorMonitor(ai, task-context)
    sRULE ← RuleCheck(ai)
    R ← λ1·sP + λ2·sB + λ3·sRULE
    decision ← PolicyEngine(R, τflag, τblock)
    Log(x, ai, sP, sB, sRULE, R, decision)
    if decision = allow then SecureGateway.execute(ai)
    else if decision = flag then SecureGateway.hold(ai)
    else reject(ai)
end for
```

V. EVALUATION PLAN

Since implementation and integration of the core modules are in progress at the time of writing, this section specifies the evaluation methodology that will be applied once the prototype is complete, so that results can be reported with the same rigor used in the surveyed literature.

A. Datasets and Attacks

We plan to evaluate the Prompt Analyzer and end-to-end pipeline using the seven natural-language task types and corresponding benchmark datasets used in prior prompt-injection evaluation work (duplicate sentence detection, grammar correction, hate detection, natural language inference, sentiment analysis, spam detection, and text summarization) [7], together with the AgentDojo benchmark suite, which additionally exercises multi-step agent tool-use scenarios rather than single-turn classification. We will evaluate against both heuristic-based attacks (naive injection, context ignoring, escape characters, fake completion,

combined attack) and optimization-based attacks, consistent with the attack taxonomy established by Liu et al. [7] and used to evaluate DataSentinel [2].

B. Baselines

We will compare AgentShield AI's detection stage against known-answer detection [7] and DataSentinel [2] as detection-only baselines, and against a privilege-list-only configuration modeled on Progent [4] and a static-rule-only configuration modeled on AgentSpec [3] as enforcement-only baselines, in order to isolate the benefit of combining detection and enforcement in a single hybrid pipeline versus deploying either mechanism alone.

C. Metrics

Consistent with standard practice in this literature [2], [7], we will report False Positive Rate (FPR), the fraction of benign actions incorrectly blocked or flagged, and False Negative Rate (FNR), the fraction of malicious actions incorrectly allowed. We will additionally report enforcement latency, the added end-to-end time per agent action introduced by the pipeline, and audit completeness, the fraction of enforcement decisions for which the dashboard can attribute the contributing signal (sP, sB, or sRULE).

D. Expected Outcomes

- A working prototype capable of detecting prompt injection and jailbreak attempts in real time, evaluated on the datasets and attacks above.
- A quantitative comparison of FPR/FNR against the detection-only and enforcement-only baselines described above.
- A measurement of enforcement latency to assess the practical overhead of the pipeline relative to unprotected agent execution.
- A demonstration of the audit dashboard's ability to explain enforcement decisions for both true-positive and false-positive cases.

VI. DISCUSSION AND LIMITATIONS

The hybrid risk-scoring design in Section IV-D is intended to avoid a single point of probabilistic failure: because sRULE is deterministic, a known high-risk action pattern is never solely dependent on an LLM's judgment, in the same spirit as combining a fine-tuned detector with rule-based checks in DataSentinel [2]. However, this also means the framework's effectiveness against genuinely novel attack patterns not covered by sRULE, and not resembling any pattern the Prompt Analyzer or Behavior Monitor were exposed to, is bounded by the quality of sP and sB alone, an inherent limitation shared with any detector that has not been explicitly hardened against adaptive attacks through a process such as the minimax fine-tuning used in DataSentinel [2].

A second limitation concerns benign instructions embedded in legitimate data, for example a user-provided document that itself contains formatting instructions for the agent to follow. As with detection-based defenses more generally, such cases can be mistaken for injected instructions, and mitigating this will likely require the Behavior Monitor to incorporate broader task context, such as conversation history, rather than judging a single action in isolation.

Finally, the enforcement thresholds τ_{flag} and τ_{block} introduce a tunable trade-off between usability and security: setting them too conservatively increases false positives and disrupts legitimate agent workflows, while setting them too permissively reduces protection. Selecting appropriate thresholds empirically, once real usage and attack data are available from the evaluation described in Section V, is left as part of the ongoing implementation work.

VII. CONCLUSION AND FUTURE WORK

This paper has presented AgentShield AI, a real-time runtime security framework for autonomous AI agents that unifies prompt analysis, continuous behavioral monitoring, hybrid LLM-and-rule-based risk scoring, and deterministic policy enforcement into a single auditable pipeline. Unlike prior work, which addresses detection, access control, or runtime enforcement as separate concerns, AgentShield AI positions all of these functions between the user's prompt and the agent's action space, so that a risky action is scored and gated before it executes rather than merely flagged after the fact. Current work is focused on implementing and integrating the core pipeline components described in Section IV; future work will report the empirical results from the evaluation plan in Section V, extend the Behavior Monitor to incorporate multi-turn conversation context, and explore adaptive fine-tuning of the Prompt Analyzer against optimization-based attacks, following the minimax methodology demonstrated by DataSentinel [2], to harden the framework against attackers with knowledge of its internal design.

REFERENCES

[1] OWASP Foundation, “OWASP Top 10 for LLM Applications 2025,” OWASP, 2025.

[2] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks,” IEEE Symposium on Security and Privacy (S&P), 2025.

[3] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents,” arXiv:2503.18666, 2025.

[4] “Progent: Securing AI Agents with Privilege Control,” arXiv:2504.11703, 2025.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” International Conference on Learning Representations (ICLR), 2023.

[6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in AISec, 2023.

[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and Benchmarking Prompt Injection Attacks and Defenses,” in USENIX Security Symposium, 2024.

[8] OpenAI, “GPT-4 Technical Report,” arXiv preprint arXiv:2303.08774, Mar. 2023.